DBMS PROJECT

Design and Implementation of a Database Management System for a Residential Society

Resident & Maintenance Billing System



Presented by: Krish

Roll No.: 25WU0102333

Team: AIML Rhinos

Project No.: 42

Introduction

- Residential societies manage hundreds of flats, residents and monthly maintenance charges.
- Records like resident details, bills and payments must stay accurate and consistent.
- A DBMS stores all society data centrally with controlled, secure access.
- Our project automates resident records and maintenance billing for a society office.



One central database for residents, flats, bills and payments.

Objectives

1 Centralized Data

Store resident, flat and staff data in one normalized database.

2 Automated Billing

Generate monthly maintenance bills without manual calculation.

3 Payment Tracking

Record payments and instantly identify pending dues.

4 Fast Queries

Answer society-office queries in seconds using SQL.

Existing System & Its Limitations

How it works today

- Paper registers and Excel sheets
- Bills calculated by hand each month
- Receipts written manually
- Dues tracked by memory or loose notes

Problems

- Errors in bill amounts and totals
- Duplicate and inconsistent records
- No quick way to find defaulters
- Data loss risk and no backups

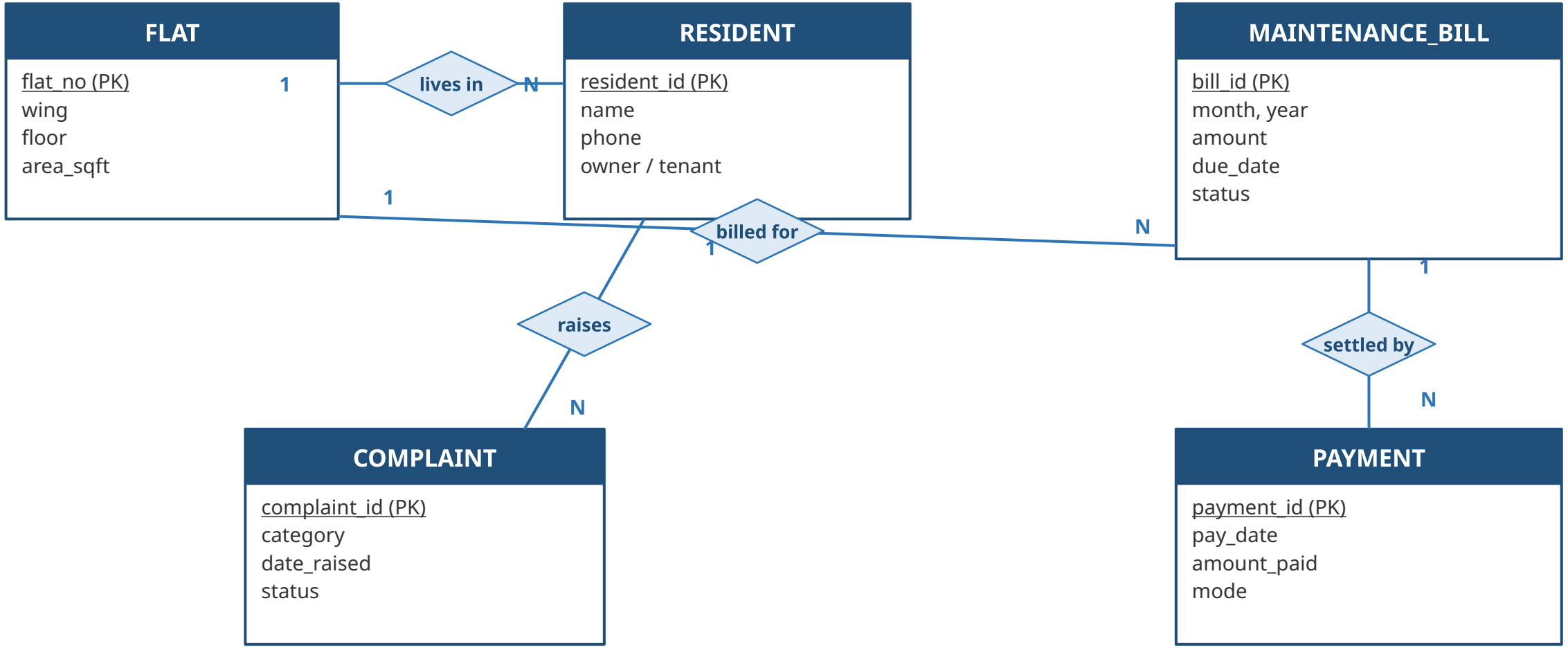
Proposed System

- Relational database (MySQL) with normalized tables up to 3NF.
- Modules: Residents, Flats, Billing, Payments, Complaints.
- Automatic monthly bill generation per flat.
- Due lists, receipts and reports via SQL queries and views.
- Role-based access for admin and office staff.



From paper registers to a single, consistent source of truth.

ER Diagram


FLAT
flat_no (PK)
wing
floor
area_sqft
1
lives in
N
RESIDENT
resident_id (PK)
name
phone
owner / tenant
1
billed for
1
N
MAINTENANCE_BILL
bill_id (PK)
month, year
amount
due_date
status
1
settled by
N
raises
N
COMPLAINT
complaint_id (PK)
category
date_raised
status
PAYMENT
payment_id (PK)
pay_date
amount_paid
mode

Database Schema (Tables)

Table	Key Attributes	Purpose
FLAT	flat_no (PK), wing, floor, area_sqft	Every flat in the society
RESIDENT	resident_id (PK), name, phone, type, flat_no (FK)	Owners and tenants per flat
MAINTENANCE_BILL	bill_id (PK), flat_no (FK), month, amount, due_date, status	Monthly bill per flat
PAYMENT	payment_id (PK), bill_id (FK), pay_date, amount_paid, mode	Payments against bills
COMPLAINT	complaint_id (PK), resident_id (FK), category, status	Resident service requests

Referential integrity: every bill belongs to a flat, every payment to a bill — enforced with foreign keys (ON DELETE / UPDATE rules).

Implementation

Tech stack

MySQL
Database & SQL queries

Python
Backend logic & connectivity

Tkinter / Web UI
Office screens & forms

SQL Views
Due lists & reports

Sample SQL

```
-- Defaulters for June 2026
SELECT f.flat_no, r.name, b.amount
FROM maintenance_bill b
JOIN flat f ON f.flat_no = b.flat_no
JOIN resident r ON r.flat_no = f.flat_no
WHERE b.status = 'UNPAID'
  AND b.month = 6 AND b.year = 2026;
```

Also implemented:

- INSERT / UPDATE forms for residents & bills
- Triggers to update bill status on payment
- Views for monthly collection summary

Results & Benefits

Error-free bills

Amounts computed from flat data — no manual arithmetic.

Instant due lists

Defaulter reports generated in one query.

Consistent records

One normalized source, no duplicate registers.

Secure access

Only authorized staff can view or modify data.

The society office now runs billing, receipts and reports from one reliable database.

Conclusion

- A normalized relational database replaces error-prone manual society records.
- Automated maintenance billing saves office time every month.
- The design extends easily to amenities, staff payroll and visitor logs.
- Learned: ER modeling, normalization, keys, triggers and SQL reporting.



Thank You! — Krish | Roll No. 25WU0102333 | Team AIML Rhinos | Project 42